Central Langfuse Platform

Shared LLM observability, agent evaluation, and prompt management for every team

Author: shivank.sharma@wipro.com (AI-Council Switzerland)

Index

This overview starts with agentic observability (build-time vs runtime), then platform infrastructure, then the three capability modules.

1. Introduction
2. Agentic Observability
3. Core Features and Infra
4. Observability
5. Evaluations
6. Prompt Management
7. How Teams Use This Platform
8. Key Business Benefits
9. Target Audience of Product
10. Conclusion

Introduction

The Central Langfuse Platform packages open-source Langfuse (version 4.4.0) as an owned, self-hosted product that the central platform team runs once for the whole organization.

Agent systems need observability across build-time checks and live runtime behavior. This product focuses mainly on runtime: shared tracing and online scoring on Central Langfuse. Build-time datasets and experiments use the same instance before you ship.

Application teams do not deploy their own Langfuse stacks. They connect with project API keys. OpenAI is the standard AI resource for application generation and for LLM-as-a-Judge on the central instance.

Agentic Observability

Agentic observability means you can see what multi-step agents did—plans, tools, LLM calls—and improve them with scores. Agent failures such as wrong tools, loops, or goal drift often appear only when the agent runs live.

Two phases:

- Build-time (pre-ship) — datasets, offline experiments, code evaluators, OpenAI LLM-as-a-Judge, and optional CI gates on fixed cases
- Runtime (primary focus) — live traces and online scores on Central Langfuse (LLM-as-a-Judge, code checks, user thumbs)

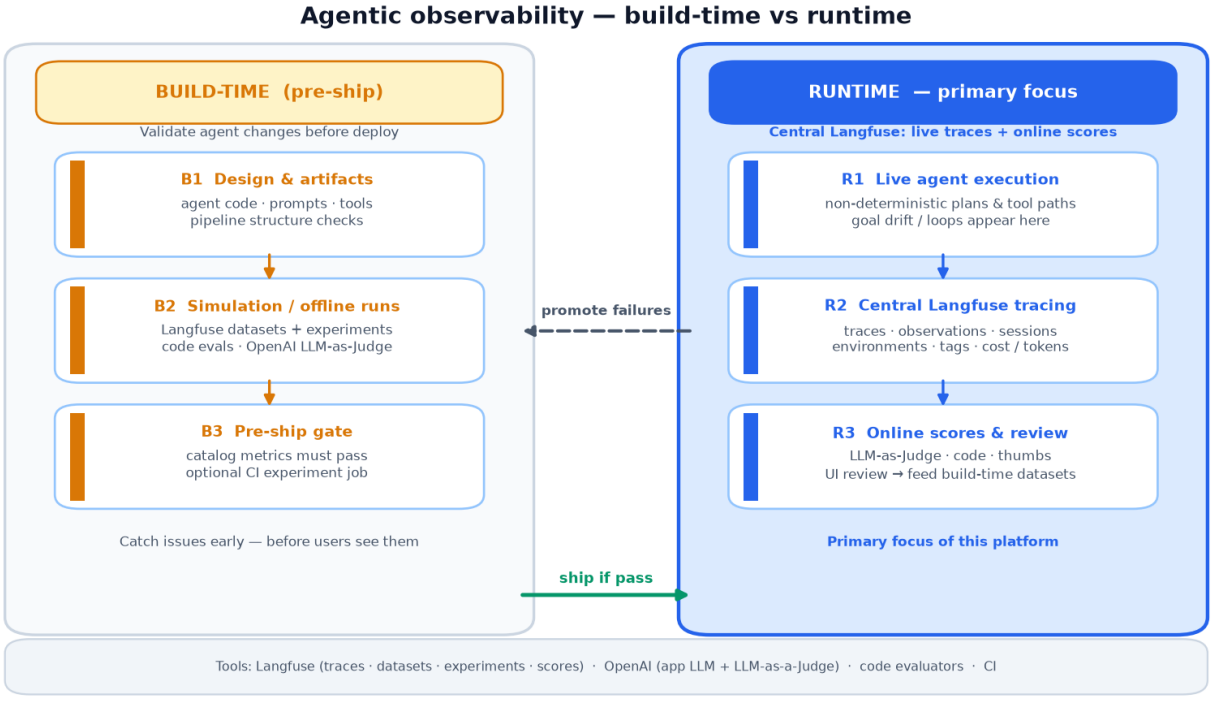
We focus mainly on runtime. Langfuse is where teams export nested agent traces, attach scores, and review production or staging behavior. Build-time experiments still matter: they run on a curated dataset and gate prompt, model, or tool changes before ship.

How the dataset is created (the build-time box in the diagram):

- Seed (gold cases) — create a Langfuse dataset per major agent or use case (for example support-agent-core). Add a first batch of realistic scenarios by hand in the UI or SDK: typical user goals, known edge cases, and acceptance checks. For tool agents, store structured input (user_goal) and optional expected_output (expected_tools, answer_contains) so code evaluators can assert path and outcome.
- Grow from runtime (promote failures) — the main ongoing source. Review live traces and online scores on Central Langfuse (failed judge, bad tool path, thumbs-down, loops). Open the failing trace, then add a dataset item that captures that scenario—copy the user goal into input; optionally set expected tools or expected answer once the correct behavior is known. Prefer adding from the Langfuse UI so the item stays linked to real production context; the SDK can create items too.
- Use at build-time — before shipping a change, run an offline experiment: your agent as the task over every dataset item, scored with catalog metrics (goal_achieved, tool_correctness, trajectory_ok, faithfulness, and so on). Ship only if the metrics gate passes. Optional CI can fail the build on regressions.

Org rules: one dataset per use case; start small (about 10–30 gold items) and grow from recurring hard failures; use the same score names online and offline; do not mix unrelated products in one dataset.

Details: docs/architecture/product-intent.md (How the dataset is created), docs/evaluation/datasets-and-experiments.md, docs/observability/overview.md.



Build-time gates changes; runtime on Central Langfuse is the primary focus

Core Features and Infra

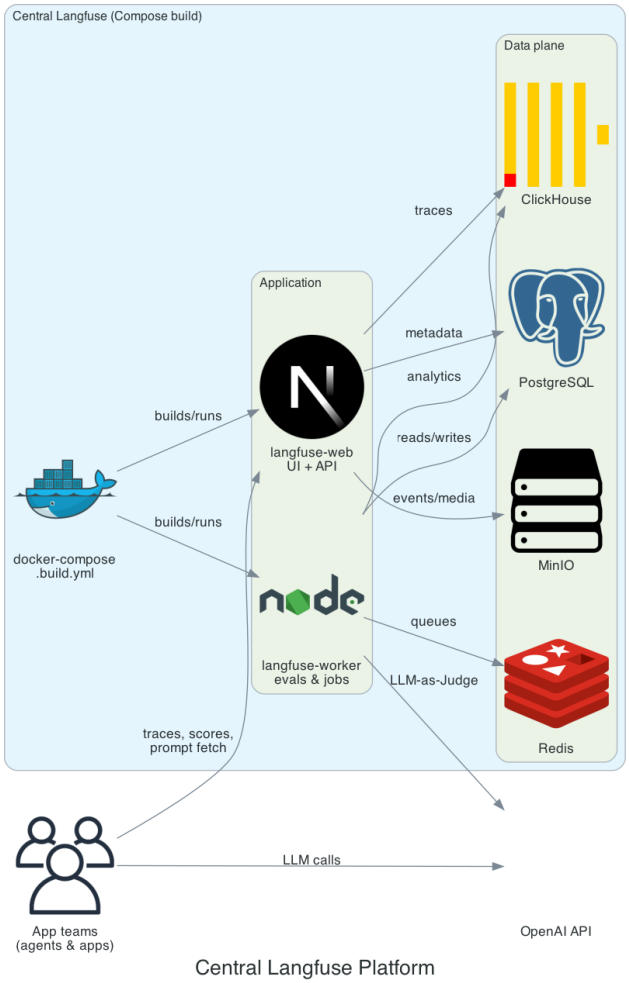
The platform is the shared Langfuse runtime that makes the observability above possible. Capabilities and data plane:

- langfuse-web (UI + API) and langfuse-worker (async jobs and eval execution), built from local Dockerfiles via docker-compose.build.yml
- PostgreSQL for application metadata; ClickHouse for analytics and trace storage; Redis for queues and cache; MinIO for S3-compatible object storage
- OpenAI API for application LLM calls and in-product LLM-as-a-Judge
- Headless project bootstrap via LANGFUSE_INIT_* for org, project, admin user, and API keys

Modules enabled by this infra (detailed in the following sections):

- Observability — traces, sessions, environments, tags, cost/tokens
- Evaluations — agent metrics, datasets, experiments, judges
- Prompt Management — versioned prompts, labels, link to traces

Operators start the stack with scripts/org/up.sh after copying .env.example. The diagram below shows how app teams, Compose-built Langfuse, and OpenAI connect.



High-level platform architecture

Observability

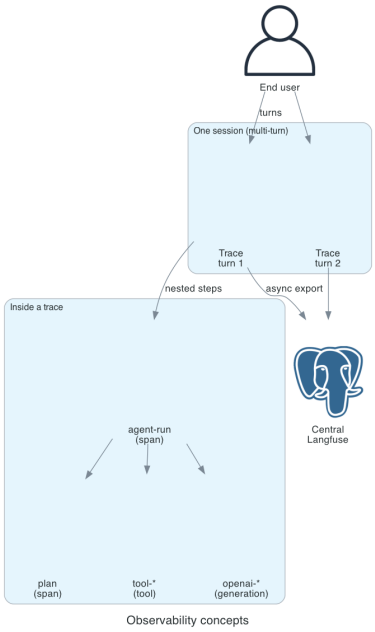
Observability is application tracing for LLM and agent systems: structured records of each request so teams can see prompts, responses, tool calls, latency, and cost. This is the runtime golden path on Central Langfuse.

Core concepts on the central instance:

- Trace — one unit of work (chat turn, agent run, or pipeline execution)
- Observation — a nested step (span, generation for LLM calls, tool steps)
- Session — groups multi-turn conversations for session replay
- Environment — dev, staging, or production so test data does not pollute live views
- Tags and metadata — always tag team and service; metadata for request IDs and flags

Why it matters for agents: multi-step failures only show up when tools and generations nest under one agent-run with stable names and clear root input/output. SDKs export asynchronously (flush on short-lived jobs).

Guidelines: docs/observability/ (get-started, data-model, best-practices, instrumentation, sessions, environments).



Sessions group traces; each trace nests spans, tools, and generations

Evaluations

Evaluations turn agent quality into measurable scores on the same central project—attached to traces, observations, sessions, or dataset runs.

Build-time: run the agent on a Langfuse dataset (experiments), score with code evaluators and/or OpenAI LLM-as-a-Judge, gate on catalog metrics, then ship if pass.

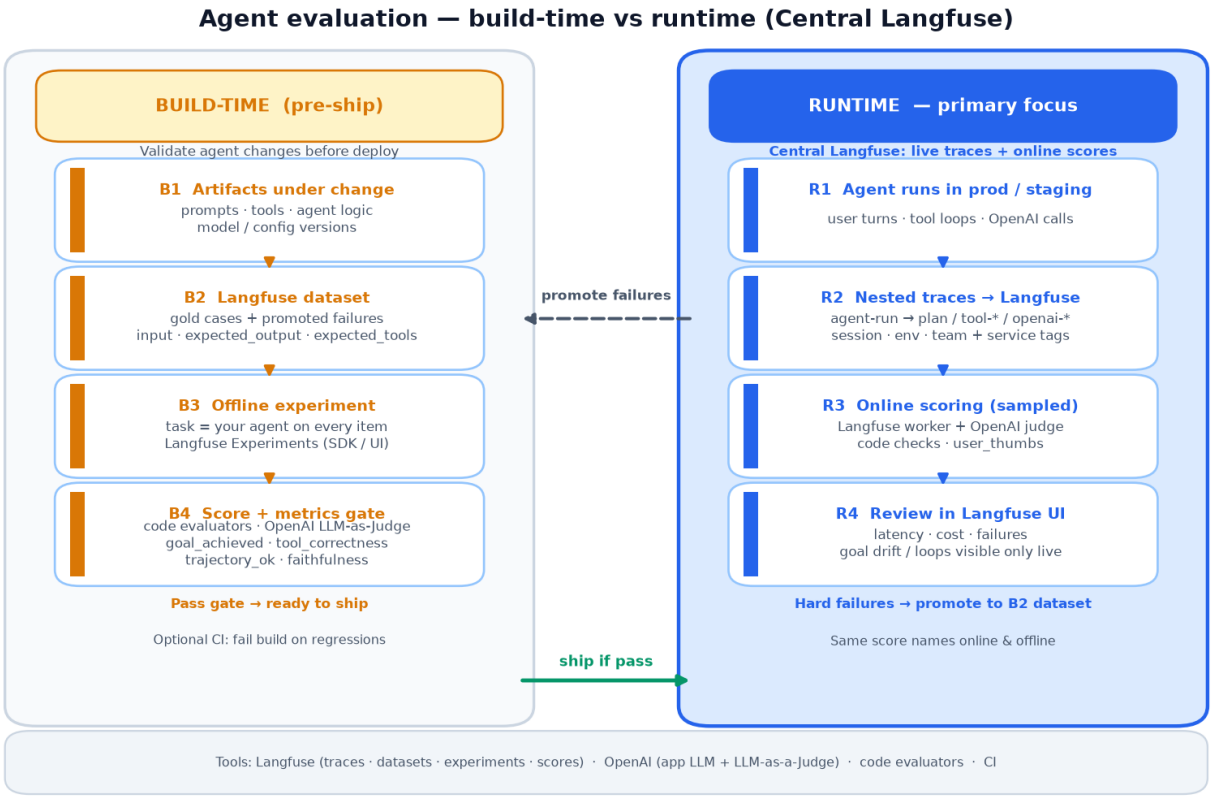
Runtime (primary): sample live traces on Central Langfuse; score online (judge, code, user thumbs); promote failures into the dataset for the next offline run.

Agent strategies:

- Final response (black-box) — goal achieved?
- Trajectory (glass-box) — right tools and path?
- Single step (white-box) — unit-test one decision

Org score catalog: goal_achieved, tool_correctness, trajectory_ok, faithfulness, answer_relevance, harmful_content, latency_budget, user_thumbs, and more (docs/evaluation/score-catalog.md).

Guidelines: docs/evaluation/ (agent-evaluation, score-catalog, datasets-and-experiments, llm-as-judge-and-openai).



Build-time experiments gate ship; runtime online scores on Langfuse promote failures back into the dataset

Prompt Management

Prompt management stores and versions prompts in Langfuse so prompt iteration is decoupled from code deployment.

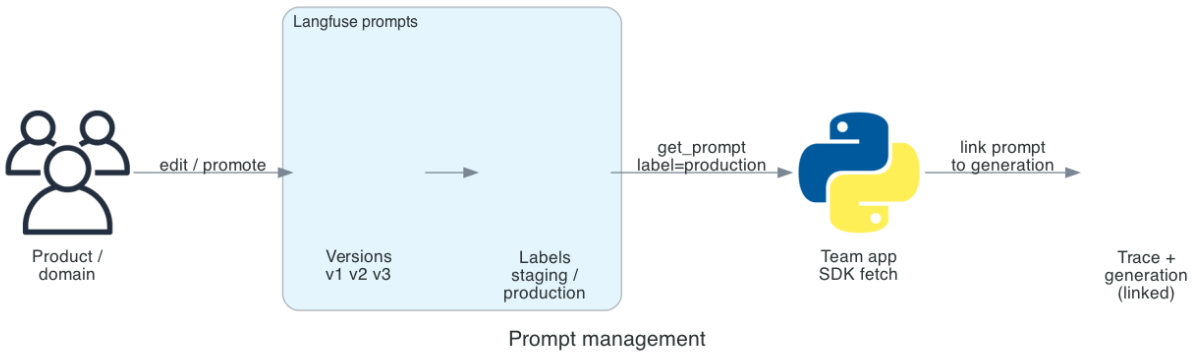
Core practices:

- Named prompts with immutable versions
- Labels such as production and staging as promotion pointers—apps fetch by name and label
- Client-side SDK caching for low-latency reads
- Link prompt versions to generations for quality analysis per version

Org rules: production services use the production label; avoid long-term hardcoded production prompts; never put secrets in templates.

Together with observability and evaluations, prompt changes can be tested offline and monitored online after label promotion.

Guidelines: docs/prompt-management/ (overview, versioning-and-labels, use-in-applications).



Version and label prompts centrally; apps fetch production and link to traces

How Teams Use This Platform

App / agent engineering teams:

- Obtain project API keys from the central platform
- Set LANGFUSE_HOST, LANGFUSE_PUBLIC_KEY, LANGFUSE_SECRET_KEY, LANGFUSE_TRACING_ENVIRONMENT, OPENAI_API_KEY
- Instrument nested traces (docs/observability) — runtime first
- Build datasets and experiments with the score catalog; use LLM-as-a-Judge where needed
- Manage prompts in Langfuse and fetch by label in app code
- Verify in the shared UI on the central host

Central platform operators:

- Own Langfuse 4.4.0 source; deploy with docker-compose.build.yml / scripts/org/up.sh
- Manage secrets, headless bootstrap, and shared OpenAI for judges
- Maintain golden-path docs under docs/

Day-to-day loop: ship traces → review failures → grow dataset → experiment before changes → promote prompt labels when metrics pass → monitor online.

Key Business Benefits

- One controlled platform for tracing, evaluation, and prompts
- Safer agent iteration via offline experiments and online scores
- Diagnosable production failures through nested traces
- Decoupled prompt updates via labels without every code release
- Owned software: Compose builds from this repository's Langfuse 4.4.0 source
- Shared standards (score catalog, tags, environments) across teams
- Self-hosted data residency with org-managed secrets

Target Audience of Product

- Central platform and SRE operators
- Application and agent engineering teams
- AI and quality owners defining metrics and judges
- Product and domain experts iterating prompts
- Engineering leads driving org golden paths
- AI-Council Switzerland stakeholders

Conclusion

Runtime first: Central Langfuse is the shared place for live agent traces and online scores; build-time datasets and experiments gate changes on the same instance. Infra, Observability, Evaluations, and Prompt Management are how the org operates that product.

Success: production traces with org conventions, experiments gating agent changes, prompts fetched by label, and a platform operated for AI-Council Switzerland.